

ACID Properties

ACID and BASE represent two contrasting approaches to database transaction management and consistency models. Understanding these is crucial for choosing the right database and architecture for your system.

ACID Properties (Traditional Databases)

ACID = Atomicity, Consistency, Isolation, Durability

Atomicity - A transaction is "all or nothing" — either all operations complete successfully, or none of them do. Partial completion is not allowed.

How it works:

- Database uses transaction log (write-ahead log)
- Records all changes before committing
- On failure, uses log to undo (rollback) partial changes
- Only marks transaction complete when all operations succeed

Example - Bank Transfer:

```
sql
BEGIN TRANSACTION;
    UPDATE accounts SET balance = balance - 100 WHERE account_id = 1; -- Deduct from Alice
    UPDATE accounts SET balance = balance + 100 WHERE account_id = 2; -- Add to Bob
COMMIT;
...
```

****Without atomicity:****
...

Step 1: Deduct \$100 from Alice ✓
Step 2: Add \$100 to Bob × (system crashes)
Result: \$100 disappears! 🤖

****With atomicity:****
...

Step 1: Deduct \$100 from Alice ✓
Step 2: Add \$100 to Bob × (system crashes)
Result: Entire transaction rolls back, Alice still has \$100 ✓

Consistency - Transactions move the database from one valid state to another valid state. All database rules, constraints, and triggers are enforced.

How it works:

- Database validates all constraints before committing
- If any constraint is violated, transaction is rolled back
- Database remains in a valid state at all times

Example - Inventory Management:

```
sql

CREATE TABLE inventory (
    product_id INT PRIMARY KEY,
    quantity INT CHECK (quantity >= 0) -- Constraint: can't be negative
);

-- This transaction violates consistency
BEGIN TRANSACTION;
    UPDATE inventory SET quantity = quantity - 10 WHERE product_id = 1;
    -- If quantity was 5, this would result in -5
COMMIT;

Error: CHECK constraint violated
Transaction rolled back
```

Types of consistency rules enforced:

Data type constraints:

```
sql

age INT -- Can't insert "twenty-five"
email VARCHAR(255) -- Can't exceed 255 characters
```

Domain constraints:

```
sql

CHECK (age >= 0 AND age <= 150)
CHECK (price > 0)
```

Referential integrity:

```
sql

FOREIGN KEY (customer_id) REFERENCES customers(id)
-- Can't insert order for non-existent customer
```

Unique constraints:

```
sql

UNIQUE (email) -- No duplicate emails
```

Custom business rules:

```
sql

-- Account balance can't go below minimum
CHECK (balance >= minimum_balance)
...
```

Isolation - Concurrent transactions execute independently without interfering with each other. Each transaction appears to run in isolation, even though many may execute simultaneously.

How it works:

- Locking mechanisms (row-level, table-level)
- Multi-version concurrency control (MVCC)
- Timestamps and versioning
- Deadlock detection and resolution

Trade-off: Higher isolation = better consistency, lower performance/concurrency

```
**The problem without isolation:**  
...
```

Time	Transaction A	Transaction B
T1	Read balance = \$100	
T2		Read balance = \$100
T3	Withdraw \$50	
T4		Withdraw \$30
T5	Write balance = \$50	
T6		Write balance = \$70
T7	Commit	Commit

```
Final balance: $70 (should be $20!)  
Lost $50 withdrawal from Transaction A  
...
```

```
**With isolation:**  
...
```

Time	Transaction A	Transaction B
T1	Read balance = \$100	
T2	Withdraw \$50	
T3	Write balance = \$50	
T4	Commit	
T5		Read balance = \$50 (sees A's commit)
T6		Withdraw \$30
T7		Write balance = \$20
T8		Commit

```
Final balance: $20 ✓
```

Levels of Isolation (from weakest to strongest)

1. Read Uncommitted (Lowest)

- Transactions can read uncommitted changes from other transactions
- **Problem:** Dirty reads (reading data that might be rolled back)
- **Use case:** Rarely used, only for non-critical data where performance is paramount.

```
Transaction A:                Transaction B:
UPDATE accounts
SET balance = 500
WHERE id = 1;

                                SELECT balance FROM accounts
                                WHERE id = 1;
                                -- Returns 500 (uncommitted!)

ROLLBACK;

                                -- B read data that never existed!
```

2. Read Committed (Default in most databases)

- Can only read committed data
- **Prevents:** Dirty Reads
- **Problem:** Non-repeatable reads (same query returns different results)
- **Use case:** Most applications, good balance of consistency and performance

```
Transaction A:                Transaction B:
                                SELECT balance FROM accounts
                                WHERE id = 1;
                                -- Returns 100

UPDATE accounts
SET balance = 500
WHERE id = 1;
COMMIT;

                                SELECT balance FROM accounts
                                WHERE id = 1;
                                -- Returns 500 (different!)
```

3. Repeatable Read

- Ensures same query returns same result within a transaction
- **Prevents:** Dirty reads, non-repeatable reads
- **Problem:** Phantom reads (new rows appearing)
- **Use case:** Financial applications, reporting where consistency within transaction is critical

```

Transaction A:
SELECT COUNT(*)
FROM orders
WHERE status = 'pending';
-- Returns 5

Transaction B:
INSERT INTO orders (status)
VALUES ('pending');
COMMIT;

SELECT COUNT(*)
FROM orders
WHERE status = 'pending';
-- Returns 6 (phantom row!)

```

4. Serializable (Highest)

- Strongest isolation, transactions execute as if serial (one after another)
- **Prevents:** All concurrency issues
- **Cost:** Significant performance penalty
- **Use case:** Critical financial transactions, when absolute consistency is required.

```

SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;

-- Database ensures this behaves as if transactions ran one at a time
-- No concurrent interference possible

```

Durability - Once a transaction is committed, the changes are permanent, even if the system crashes immediately after.

How it works:

- **Write-Ahead Logging :**

1. Transaction starts
2. Changes written to transaction log (on disk)
3. Log entry marked as committed
4. Only then does COMMIT return success
5. Later, changes applied to actual data files (can be delayed)

If crash occurs:

- Transaction log is replayed on restart
- Committed transactions are recovered
- Uncommitted transactions are rolled back

Mechanisms:

Transaction logs:

- All changes written to sequential log file
- Fast to write (sequential I/O)
- Persistent storage (not just memory)

Checkpoints:

- Periodically flush data from memory to disk
- Mark point in log up to which all changes are on disk

Recovery:

- On restart, replay log from last checkpoint
- Apply committed transactions
- Discard uncommitted ones

```
BEGIN TRANSACTION;  
    INSERT INTO orders (customer_id, amount) VALUES (123, 99.99);  
COMMIT; -- Returns success  
  
-- Power failure occurs 1 millisecond later  
-- System restarts  
-- Order still exists in database ✓
```

ACID Example - Complete Transaction

```
sql  
  
-- Transfer $100 from Alice to Bob  
BEGIN TRANSACTION; -- Start atomic unit  
  
-- Consistency: Check business rules  
IF (SELECT balance FROM accounts WHERE id = 1) < 100 THEN  
    ROLLBACK; -- Insufficient funds  
END IF;  
  
-- Isolation: Lock rows being modified  
UPDATE accounts  
SET balance = balance - 100  
WHERE id = 1; -- Alice's account  
  
UPDATE accounts  
SET balance = balance + 100  
WHERE id = 2; -- Bob's account  
  
-- Atomicity: Both updates or neither  
-- Consistency: All constraints checked  
COMMIT; -- Durability: Changes persisted to log  
  
-- If power fails after COMMIT:  
-- - Changes are in write-ahead log  
-- - Recovery will replay the log  
-- - Transaction will be completed on restart  
...
```

